



Institut Africain d'Informatique

Etablissement Inter-Etats d'Enseignement Supérieur

Représentation du Togo (IAI-TOGO)

07 BP 12456 Lomé 07 Tél. : (+228) 22 20 47 00

E-mail : iaitogo@iai-togo.tg Site Web : www.iai-togo.tg

Cahier d'exercices



Conception et Implémentation des Structures de Données (Python)



Année Académique 2025 - 2026

Chapitre 1 : Concepts de base et Opérateurs

Exercice 1 : Python comme une calculatrice

Prévoyez sans exécution l'évaluation des expressions arithmétiques suivantes :

```
1. 6 * 7 + 3
2. 6 * (7 + 3)
3. 45 // 7
4. 3 * 7 // 4
5. (3 * 7) // 4
6. (45 // 7) * 7 + 45 % 7
```

Exercice 2 : (Int vs Float)

1. Dans ce qui suit, on utilise le symbole ; pour séparer des instructions distinctes.
2. Effectuer les opérations suivantes.

```
10 * 10; 10 + 10; 10 - 1; 10 ** 3; 100 // 3; 100 / 3; 100 % 3
```

3. Effectuer les opérations suivantes.

```
10. * 10; 10 + 10.; 10. - 1; 10. ** 3; 100. // 3; 100 / 3.; 100. % 3
```

4. Quelles sont les différences entre les deux suites de commande précédentes ?

Exercice 3 : Chaines de caractères

1. Quelle est l'action de l'opérateur + entre deux chaines de caractères ?
2. Quelle est l'action de l'opérateur * entre une chaine de caractère et un nombre ?
3. L'expression suivante est-elle valide ? Que vaut-elle ?

```
"a" * 2 + "b" * 3 + "c" * 4
```

4. Les opérateurs - et / peuvent-ils être utilisés avec les chaines de caractères ? Pourquoi ?
5. Sans Exécuter l'instruction suivante, que vaut-elle ?

```
'ab' * 5 == 'a' + 'ba' * 4 + 'b'
```

6. On peut tester l'égalité entre deux chaines de caractères comme le montre la question précédente. Qu'en est-il des opérateurs de comparaisons <, >, etc. .

Exericice 4 : La table de vérité

On considère l'expression logique $(a \text{ or } b) \text{ and } a$ Quelle est sa table de vérité ?

A		
a	b	$(a \text{ or } b) \text{ and } a$
False	False	False
False	True	True
True	False	True
True	True	True

B		
a	b	$(a \text{ or } b) \text{ and } a$
False	False	False
False	True	False
True	False	True
True	True	True

C		
a	b	$(a \text{ or } b) \text{ and } a$
False	False	False
False	True	False
True	False	False
True	True	True

D		
a	b	$(a \text{ or } b) \text{ and } a$
False	False	False
False	True	False
True	False	False
True	True	False

Exericice 5 : Équation

L'équation suivante est-elle vraie ?

$$\frac{10^{10} + 10^{-10}}{10^{10}} = 1$$

Exericice 6 : Devinette

Devine le résultat des programmes suivants sans les exécuter dans Python.

1.

```
1. x = 5
2. y = 10
3. result = (x > 3) and (y < 20)
4. print(result)
```

2.

```
1. a = True
2. b = False
3. c = True
4. result = a and b or c
5. print(result)
```

3.

```
1. x = 8
2. y = 12
3. result = (x % 2 == 0) and (y % 3 == 0)
4. print(result)
```

4.

```
1. x = 20
2. y = 30
3. result = (x > 10) and (y < 25) or (x + y == 50)
4. print(result)
```

5.

```
1. x = 15
2. y = 25
3. result = (x < 10) or (y > 20)
4. print(result)
```

6.

```
1. a = True
2. b = False
3. result = (a and b) or (not a and not b)
4. print(result)
```

Exercice 7 : Expressions booléennes et conditions

1. Admettons que la tranche d'âge pour pouvoir monter sur une montagne russe dans un parc d'attraction est entre 10 et 60 ans (inclus), et que Thomas est âgé de 12 ans. Créez une variable `age` initialisée à 12, et écrivez une expression qui est égale à `True` si Thomas peut monter sur l'attraction, et `False` sinon.

En vous inspirant de votre solution à l'exercice précédent, écrivez une expression qui est égale à "yes" si Thomas a le droit de monter sur la montagne russe, et "no" sinon.

2. Admettons que pour participer à un concours de photographie, il faut avoir au moins 18 ans et être titulaire d'une carte d'étudiant. Vous avez 20 ans et vous êtes étudiant.
 - Créez une variable `age` initialisée à 20.
 - Créez une variable `est_etudiant` initialisée à `True`.

Écrivez une expression qui est égale à `True` si vous pouvez participer au concours, et `False` sinon.

3. Un parc d'attractions propose des tarifs réduits pour les enfants et les seniors. Les conditions pour bénéficier de ces tarifs sont les suivantes :
 - Les enfants doivent avoir entre 4 et 12 ans inclus pour payer un tarif réduit.
 - Les seniors doivent avoir au moins 65 ans pour bénéficier d'un tarif réduit.

- Les personnes âgées de 13 à 64 ans paient le tarif plein.

Vous avez 17 ans. Vous souhaitez savoir si vous pouvez bénéficier d'un tarif réduit ou si vous devez payer le tarif plein.

- Créez une variable `age` initialisée à 17.
- Écrivez une expression qui indique si vous pouvez bénéficier d'un tarif réduit. Si vous pouvez, l'expression doit être égale à `"tarif réduit"`, sinon elle doit être égale à `"tarif plein"`.

Chapitre 2 : Les structures conditionnelles et itératives

Exercice 1 : Intervalle, nombre positif, nombre négatif

1. Écrire un programme qui permet de saisir un nombre puis déterminer s'il appartient à un intervalle donné, sachant que les extrémités de l'intervalle sont fixées par l'utilisateur.
2. Écrire un programme qui demande deux nombres à l'utilisateur et l'informe ensuite si leur produit est négatif ou positif. Attention toutefois on ne doit pas calculer le produit des deux nombres.

Exercice 2 : Évaluation de l'admissibilité d'un étudiant

Considérez un système qui évalue si un étudiant est admissible à une bourse d'études.

Les critères sont les suivants :

1. La moyenne générale (GPA) doit être supérieure ou égale à 3.0.
2. Le score du test d'aptitude doit être entre 85 et 115.
3. Les heures de service communautaire doivent être au moins de 50.

Écrivez un programme qui demande à l'utilisateur d'entrer les trois valeurs et qui affiche si l'étudiant est admissible ou non à la bourse.

Exercice 3 : Nombres premiers

Écrire un programme en python permettant de vérifier si un nombre entré par l'utilisateur est premier ou pas.

Rappel : Un nombre premier est un nombre entier naturel supérieur à 1 qui n'a aucun diviseur positif autre que 1 et lui-même.

On suivra la logique suivante :

1. Cas de base :
 - a. Si le nombre est inférieur ou égal à 1, il n'est pas premier. Par définition, les nombres premiers doivent être supérieurs à 1.
 - b. Si le nombre est inférieur ou égal à 3, il est premier.
 - c. Si le nombre est pair et supérieur à 2, il n'est pas premier (divisible par 2).
2. Cas itératif :
 - a. Itérer à partir de 3, en incrémentant de 2 à chaque fois, jusqu'à la racine carrée du nombre pour parcourir les diviseurs potentiels.
 - b. Si le nombre est divisible par l'un de ces entiers, il n'est pas premier.
 - c. Si la boucle se termine sans avoir trouvé de diviseur, le nombre est premier

Modifier le programme obtenu pour afficher tous les nombres premiers inférieur au nombre saisi par l'utilisateur.

Exercice 4 : Système de réservation de billets de cinéma

Vous êtes en charge de créer un système de réservation de billets pour un cinéma. Les règles de réservation sont les suivantes :

1. Le film est interdit aux moins de 12 ans.
2. Le tarif normal est de 5 000 francs
3. Les séances de matinée (avant 12 h) ont un tarif réduit de 4 000 francs.
4. Les étudiants bénéficient d'une réduction supplémentaire de 1500 francs sur présentation d'une carte valide.
5. Les séances après 20 h offrent un siège premium sans frais supplémentaires, normalement ce siège coûte 1000 francs de plus.

Écrivez un programme qui demande à l'utilisateur son âge, s'il est étudiant, l'heure de la séance choisie et calcule le prix final du billet en tenant compte des informations ci-dessus.

Exemple de traces d'exécution

Entrées utilisateur :

- Âge de l'utilisateur : 20
- L'utilisateur est étudiant : oui
- Heure de la séance choisie : 21

Sortie du programme :

- Le prix final de votre billet est de 3500.00 francs.

Explication :

- L'utilisateur a plus de 12 ans, donc il peut voir le film.
- L'utilisateur est étudiant, donc il bénéficie d'une réduction de 1500 francs.
- La séance est après 20h00, donc l'utilisateur a droit à un siège premium sans frais supplémentaires.
- Le tarif normal est de 5000 francs, avec la réduction étudiante, le prix devient 3500.00 francs car le siège premium est offert gratuitement.

Exercice 5 : Calcul d'heures supplémentaires

Écrire un programme qui permet de calculer le montant des heures supplémentaires d'un employé, sachant le prix unitaire d'une heure, selon le barème suivant :

- Les 39 premières heures sans supplément,
- De la 40^e à la 44^e heure sont majorées de 50 %,
- De la 45^e à la 49^e heure sont majorées de 75 %,
- De la 50^e heure ou plus, sont majorées de 100 %.

Exercice 6 : Une minute plus tard

Écrivez un programme qui lira au clavier l'heure et les minutes, et il affichera l'heure qu'il sera une minute plus tard.

Par exemple, si l'utilisateur tape 21 puis 32, le programme doit répondre : "Dans une minute, il sera 21 heures(s) 33".

NB : on suppose que l'utilisateur entre une heure valide. Pas besoin donc de la vérifier.

Chapitre 3 : Les séquences et les chaînes de caractères

Exercice 1 : Recherche d'occurrences

1. Écrivez un programme en Python qui calcule la somme de tous les nombres pairs jusqu'à un nombre qui sera saisi par l'utilisateur.
2. Écrire un programme qui compte le nombre de voyelles et de consonnes présent dans une chaîne de caractères qui sera saisie par l'utilisateur.
3. Écrire un programme qui donne le récapitulatif d'apparition de tous les caractères de l'alphabet français dans une phrase saisie par l'utilisateur.
4. Écrire un programme en Python qui doit compter le nombre d'occurrences d'une lettre particulière dans une chaîne de caractères donnée. Par exemple, pour la chaîne de caractères "Bonjour tout le monde", il doit être retourné la valeur 3 pour la lettre caractères 'o'. On demandera à l'utilisateur de saisir les deux chaînes.
5. Écrire un programme qui :
 - a. Affiche la table de multiplication d'un nombre saisi par l'utilisateur
 - b. Affiche la table de multiplication Complète de tous les nombres jusqu'au nombre saisi par l'utilisateur.

Exercice 2 : Palindrome

1. Écrire un programme qui lit une chaîne de caractères et vérifie si elle est palindrome ou non. Le palindrome, du grec palindromos, « qui revient sur ses pas », est un mot ou un groupe de mots qui peut être lu indifféremment de gauche à droite ou de droite à gauche, par exemple les mots « kayak », « rAdA R », « CoLoC », « rever », « ReSsAsSer ».
2. On souhaite gérer le cas où la chaîne est polluée (Contient des chiffres, caractères accentués, caractères d'autres langues). Dans ce cas, on ignorera ces caractères lors de la vérification.
3. Testez votre programme avec les chaînes de caractères suivantes : radar, reSsAssEr, rAd2ar : manman, m/adam7, Madam, S.o.s.

Exercice 3 : Tautogramme et Totalogramme

1. Le tautogramme est un jeu poétique où pour faire des phrases, on ne se sert que de mots commençant par la même lettre. Cela vient du grec "tautos" qui veut dire "même" et "gramma" qui signifie "lettres".

Exemple :

Ces crocodiles cruels croquent cinq cent cinquante-cinq caméléons curieux.
Vendredi, votre voisine Véronique viendra vite vous voir.
Une usine uruguayenne utilise uniquement un uranium ukrainien.

Écrire un programme qui permet à l'utilisateur de saisir une chaîne de caractère et qui dit s'il s'agit d'un tautogramme ou pas.

2. Une phrase est dite « totalogramme » si chacun de ses mots commence et se termine par la même lettre. Exemple : la phrase « Abdalla emprunte à Daoud ses souliers » est totalogramme.
 - Écrire un programme qui permet de saisir une phrase (on suppose que deux mots consécutifs sont séparés par un seul espace) puis d'afficher un message indiquant si cette phrase est « totalogramme » ou non. (Aucun contrôle de saisie n'est demandé).

Exemple :

Allal**a** **e**mpr**e**inte **t**emporairement **a** **d**aoud **s**es **s**ouliers**s**.

Exercice 4 : Les opérations sur les chaînes de caractères

Considérons la chaîne de caractères définie ci-dessous :

s = "Tellement de choses qu'on promet"

1. Que valent les expressions suivantes ?

1. s[0:9]
 2. s[0:1]
 3. s[:3]
 4. s[0:10-1]
 5. s[0:10:2]
 6. s[-5:-1]
 7. s[-1:-5]
 8. s[::3]
 9. s[-1:-10:-2]
 10. s[-1:-10:-2]
 11. s[:len(s)//2]

2. Produire la sous-chaîne suivante en vous servant du slicing et de la concaténation :

'choses promises'

3. Écrire les expressions pour réaliser les actions suivantes :
 - a. Déterminer la longueur de la chaîne s
 - b. Compter le nombre d'occurrences de la lettre "e" dans s
 - c. Vérifier si le mot "promet" est présent dans s
 - d. Déterminer l'index de la première occurrence du mot "choses" dans s
 - e. Remplacer "promet" par "accomplit" dans s
 - f. Extraire les trois derniers mots de s en utilisant .split() et une tranche?
 - g. Transformer s en majuscules et en minuscules
 - h. Supprimer les espaces au début et à la fin de s (s'il y en avait)
 - i. Vérifier si s commence par "Tellement" et se termine par "promet"

j. Diviser s en une liste de mots et afficher le mot le plus long

Exercice 5 : Les séquences

1. Que va afficher le code suivant ?

```
1. for i in range(1, 10, 3):  
2.     print(i)
```

- a. 1, 4, 7, 10
- b. 1, 4, 7
- c. 1, 3, 6, 9
- d. 1, 5, 9

2. Quelle option produit la séquence inversée 10, 8, 6, 4 ?

- a. range(10, 4, 2)
- b. range(10, 3, -2)
- c. range(10, 4, -2)
- d. range(10, 4, -1)

3. Quelle option permet d'afficher uniquement les nombres pairs de 10 à 20 inclus ?

- a. range(10, 21, 2)
- b. range(10, 20, 2)
- c. range(10, 22, 2)
- d. range(10, 19, 2)

4. Quelle est la sortie de ce code ?

```
1. for i in range(0, 10, 3):  
2.     for j in range(2):  
3.         print(i + j, end=" ")
```

- a. 0 1 3 4 6 7 9 10
- b. 0 1 3 4 6 7
- c. 0 1 3 4
- d. 0 3 6 9

5. Quelle est la sortie du code suivant ?

```
1. for i in range(0, 10, 3):  
2.     for j in range(i):  
3.         print(i + j, end=" ")
```

6. Complétez le code pour parcourir la séquence : -10, -7, -4, -1

Exercice 6 : Nombres parfaits et nombres amis

1. Nombres Parfaits

```
1. for i in range(.....):  
2.     print(i)
```

Écrire une fonction nommée `estParfait` qui prend en paramètre un nombre entier strictement positif donné, retourne `True` si ce nombre est parfait et `False` dans le cas contraire.

Un nombre est dit parfait s'il est égal à la somme de ses diviseurs stricts.

Exemple : $28 = 1 + 2 + 4 + 7 + 14$

Écrire une fonction qui affiche la suite de tous les nombres parfaits inférieurs ou égaux à un nombre entier positif saisi par l'utilisateur, noté n .

Voici la liste des nombres parfaits inférieurs à 10 000 : 6, 28, 496, 8128

2. Nombres amis

Deux nombres entiers n et m sont qualifiés d'amis, si la somme des diviseurs de n est égale à m et la somme des diviseurs de m est égale à n (on ne compte pas comme diviseur le nombre lui-même et 1).

Exemple : les nombres 48 et 75 sont deux nombres amis puisque :

Les diviseurs de 48 sont : $2 + 3 + 4 + 6 + 8 + 12 + 16 + 24 = 75$

Les diviseurs de 75 sont : $3 + 5 + 15 + 25 = 48$.

Écrire une fonction qui permet de déterminer si deux entiers n et m sont amis ou non.

Exercice 7 : Une légende

Une légende de l'Inde ancienne raconte que le jeu d'échecs a été inventé par un vieux sage, que son roi voulut remercier en lui affirmant qu'il lui accorderait n'importe quel cadeau en récompense. Le vieux sage demanda qu'on lui fournisse simplement un peu de riz pour ses vieux jours et plus précisément un nombre de grains de riz suffisant pour que l'on puisse en déposer 1 seul sur la première case du jeu qu'il venait d'inventer, deux sur la suivante, quatre sur la troisième et ainsi de suite jusqu'à la 64^e case.

Écrivez un programme Python qui affiche le nombre de grains à déposer sur chacune des 64 cases du jeu. Calculez ce nombre de deux manières :

1. Le nombre exact de grains (nombre entier)
2. Le nombre de grains en notation scientifique (nombre réel)

Exercice 8 : Manipulation de chaîne de caractères

1. Pouvez-vous décrypter le nom de mon crush ?

```
0x2c 0x3d 0x23 0x31 0x68 0x6f 0x65 0x6c 0x76 0x76 0x72  
0x73 0x70 0x6c 0x23 0x6a 0x71 0x6c 0x6b 0x77 0x72 0x71  
0x23 0x2f 0x77 0x75 0x64 0x68 0x6b 0x23 0x77 0x71 0x64  
0x6c 0x6f 0x64 0x79 0x23 0x77 0x44
```

Exercice 9 : Cryptographie

On veut compresser une chaîne de caractères données `s` dont la taille ne dépasse pas 50 caractères en une chaîne résultat compressé de la manière suivante : parcourir la chaîne `s` de gauche à droite en comptant le nombre d'occurrence successives de chaque caractère de la chaîne `s`, puis de ranger la chaîne compressé, ce nombre suivi du caractère en question.

Écrire un programme Python permettant de saisir la chaîne `s` qui doit être non vide et formée uniquement par des lettres alphabétiques, puis de former et d'afficher la chaîne compressé selon le principe décrit précédemment.

Exemple :

Si `s = 'aaaFyBssssssssssssazz'` alors la chaîne compressé qui sera affichée est `'3a1F1y1B12s1a2z'`.

Exercice 1 : Recherche d'occurrences

1. On considère la fonction `foo` dont la définition est donnée ci-dessous. Que renvoie `foo("100")` ?

```
1. def foo(s):  
2.     a=""  
3.     for i in range(0, len(s), 1):  
4.         a = a + "A"  
5.     return a
```

2. Écrire une fonction `cube` qui prend un entier n en argument et qui renvoie la somme des n premiers cubes si $n \geq 5$, et 0 sinon. Ainsi, `cube(5)` vaudra 225 (soit $1^3 + 2^3 + 3^3 + 4^3 + 5^3$) tandis que `cube(-5)` vaut 0.
3. Écrire une fonction `nbrDiviseurs` qui prend un entier naturel strictement positif comme argument et renvoie le nombre de ses diviseurs. Par exemple, `nbrDiviseurs(10)` renverra 4 car 10 a quatre diviseurs qui sont 1, 2, 5 et 10.
4. Écrire une fonction nommée `premier` qui prend un entier naturel strictement positif comme argument et renvoie le booléen `True` si l'entier est premier et `False` sinon.

Exercice 2 : Énigme familiale

On souhaite utiliser un programme pour résoudre l'énigme suivante :

J'ai quatre fois l'âge de mon petit-fils. Si on inverse les deux chiffres de chaque âge, le nouvel âge de mon petit-fils est égal à trois fois le mien.

1. Écrire une fonction nommée `inverse(age)` qui prend en paramètre un entier `age` comprise entre 10 et 99 et qui retourne le nombre que l'on obtient en permutant les deux chiffres de `age`. Par exemple, `inverse(42)` retourne 24.

On rappelle que l'opération modulo, notée `%`, permet de calculer le reste de la division entière. Ainsi, par exemple, 7 modulo 3, noté `7 % 3` en Python, vaut 1.

2. Écrire une fonction `est_une_reponse` qui prend en paramètre un entier `age_grand_pere` compris entre 10 et 99 représentant l'âge du grand-père et un entier `age_petit_fils` compris entre 10 et 99 représentant l'âge du fils. Cette fonction retourne `True` si et seulement si ces deux entiers sont une réponse valide à l'énigme décrite plus haut.
3. Écrire une fonction `resoudre_enigme(age_grand_pere)` qui prend en paramètre un entier `age_grand_pere` compris entre 10 et 99 représentant l'âge du grand-père et qui retourne, s'il existe, un entier `age_petit_fils` compris entre

4. 10 et 99 tel que `est_une_reponse(age_grand_pere, age_petit_fils)` renvoie `True`. S'il n'existe pas un tel nombre, cette fonction

renvoie `False`. (Indication : Il suffit d'essayer tous les âges compris entre 10 et 99.)

5. Écrire une fonction `resoudre_age_petit_fils(age_grand_pere)` qui prend en paramètre un entier `age_grand_pere` compris entre 10 et 99 représentant l'âge du grand-père et qui retourne, s'il existe, un entier `age_petit_fils` compris entre 10 et 99 tel que `resoudre_enigme(age_grand_pere, age_petit_fils)` renvoie `True`. S'il n'existe pas un tel nombre, cette fonction renvoie `False`. (Indication : Il suffit d'essayer tous les âges compris entre 10 et 99.)
6. Proposer un cas d'utilisation de vos fonctions.

Exercice 3 : Jardinage

Un jardinier veut se lancer dans la culture de tomates. Un type de tomates est donné par un facteur f . Chaque année une graine plantée de tomates donne alors $f + 3$ tomates et chaque tomate contient $5 \times f$ graines.

1. Écrire une fonction `nbreTomates` qui prend comme arguments un entier f correspondant à un type de tomates et un entier g correspondant à un nombre de graines initiales et qui renvoie le nombre tomates obtenues au bout d'un an pour le nombre de graines.

Par exemple, `nbreTomates(2, 3)` renvoie 15.

2. Écrire une fonction `nbreGraines` qui prend comme arguments un entier f correspondant à un type de tomates et un entier g correspondant à un nombre de graines initiales et qui renvoie le nombre de graines obtenues au bout d'un an. On compte ici le nombre de graines fournies par les nouvelles tomates, les graines initiales ayant été plantées.

Par exemple, `nbreGraines(2, 3)` renvoie 150 car on a obtenu 15 tomates fournissant chacune 10 graines.

3. Écrire une fonction `nbreTomAnnee` qui prend comme arguments un entier f correspondant à un type de tomates, un entier g correspondant à un nombre de graines initial et un entier n correspondant à un nombre d'années et qui renvoie le nombre de tomates obtenu l'année n , sachant que chaque année on plante les nouvelles graines des tomates obtenues l'année précédente. Par exemple, si on a 3 graines d'un type de tomates de facteur 2. Au bout d'un an on a 150 graines, donc la deuxième année on aura 750 tomates qui produiront 7500 graines et la troisième année on aura alors 37500 tomates. Ainsi `nbreTomAnnee(2,3,2)` renvoie 750 et `nbreTomAnnee(2,3,3)` renvoie 37500.
4. Le jardinier hésite entre acheter 4 graines d'un type de tomates avec un facteur 2 ou acheter 2 graines d'un type de tomates avec un facteur 4. Il voudrait savoir avec quel choix il aura le plus de tomates la dixième année. Écrire un programme utilisant les fonctions précédentes et qui affiche

le message "Choix 1 meilleur" si la dixième année le premier choix produit plus de tomates et sinon le programme affiche "Choix 2 meilleur".

Exercice 1 : Traitement de notes

Le fichier notes.txt joint à cet exercice contient les notes des étudiants d'une université donnée sur lequel on souhaite effectuer un certain nombre d'opérations.

Le format des données est comme suit :

- Chaque ligne correspond à un étudiant.
- Le nom de l'étudiant est suivi de ses notes, séparées par des barres obliques (/).
- Les notes sont sous forme de nombres décimaux ou entiers, représentant les résultats obtenus dans différentes matières.

Exemple : Laurent 14.0/10.25/12/09/14 → Laurent a cinq notes : 14.0, 10.25, 12, 9 et 14.

Implémentez les fonctions ci-dessous :

```
lire_fichier(nom_fichier: str) -> List[str]:
    Lit le contenu d'un fichier texte et retourne une liste de lignes.
-----
extraire_nom(ligne: str) -> str:
    Extrait le nom d'une ligne de texte.
-----
extraire_notes(ligne: str) -> List[float]:
    Extrait les notes d'une ligne de texte et les retourne sous forme de liste de flottants.
-----
afficher_notes(etudiant: str) -> None:
    Affiche le nom d'un étudiant et ses notes.
-----
calculer_moyenne(notes: List[float]) -> float:
    Calcule la moyenne des notes.
-----
calculer_ecart_type(notes: List[float]) -> float:
    Calcule l'écart type des notes.
-----
afficher_statistiques(etudiant: str) -> None:
    Affiche les statistiques d'un étudiant : moyenne et écart type.
-----
moyenne_generale(etudiants: List[str]) -> float:
    Calcule la moyenne générale de tous les étudiants.
-----
moyenne_maximale(etudiants: List[str]) -> float:
    Calcule la moyenne maximale parmi tous les étudiants.
-----
moyenne_minimale(etudiants: List[str]) -> float:
    Calcule la moyenne minimale parmi tous les étudiants.
```


Exemple d'exécution

```
-----  
Etudiant : Laurent  
Note n°1 : 14.0  
Note n°2 : 10.25  
Note n°3 : 12.0  
Note n°4 : 9.0  
Note n°5 : 14.0  
-----
```

```
Etudiant : Awa  
Note n°1 : 11.5  
Note n°2 : 13.0  
Note n°3 : 15.0  
Note n°4 : 8.0  
Note n°5 : 10.0  
-----
```

```
Statistiques pour Laurent :  
Moyenne : 11.85  
Ecart type : 2.00  
-----
```

```
Statistiques pour Awa :  
Moyenne : 11.50  
Ecart type : 2.41  
-----
```

```
Moyenne générale : 11.68  
Moyenne maximale : 11.85  
Moyenne minimale : 11.50
```

Exercice 2 : Adresses IPV4 et IPV6

Etant donné un fichier texte nommé 'IPV4.txt' contenant dans chaque ligne une adresse IPV4. On se propose de vérifier la validité des adresses IPV4 stockés dans ce fichier, de déterminer la classe à laquelle appartient chacune des adresses valides, de les faire migrer vers le système IPV6 et de stocker dans un fichier d'enregistrements nommé 'IPV6.txt' chaque adresse IPV4 valide ainsi que la classe à laquelle elle appartient et son équivalent en IPV6.

Pour ce faire, on dispose des informations suivantes :

1. Une adresse IPV4 valide est codée sur quatre octets (32 bits) et représentée sous la forme X.W.Y.Z avec W, Y, Y et Z sont quatre entiers naturels appartenant chacun à l'intervalle [0,255] et séparés par le caractère ' . '

NB : Pour vérifier la validité d'une adresse IPV4, vous êtes appelé uniquement à vérifier si W, X, Y et Z sont dans l'intervalle [0, 255].

2. Chaque adresse IPV4 valide appartient à une classe :

- Classe A, si la valeur du premier bit à gauche de la représentation en binaire de W est 0.
- Classe B, si la valeur des deux premiers bits à gauche de la représentation en binaire de W est 10.
- Classe C, si la valeur des trois premiers bits à gauche de la représentation en binaire de W est 110.
- Classe D, si la valeur des quatre premiers bits à gauche de la représentation en binaire de W est 1110.
- Classe E, si la valeur des quatre premiers bits à gauche de la représentation en binaire de W est 1111.
- Une adresse IPV6 est codée sur 16 octets (128 bits). Pour faire migrer une adresse IPV4 valide vers le système IPV6, on va s'intéresser uniquement au bloc de 32bits dans l'adresse IPV6 qui représente la conversion en hexadécimal de l'adresse IPV4. Pour ce faire, on convertit chacun des nombres W, X, Y et Z en hexadécimal, puis, les concaténer en insérant le caractère ':' au milieu du résultat obtenu.

Exemple :

L'adresse 155 . 105 . 50 . 68 est valide et elle appartient à la classe B, car la valeur des deux premiers bits à gauche de la représentation en binaire de 155 qui est 10011011 est 10.

- L'équivalent du nombre décimal 155 en hexadécimal est 9B
- L'équivalent du nombre décimal 105 en hexadécimal est 69
- L'équivalent du nombre décimal 50 en hexadécimal est 32
- L'équivalent du nombre décimal 69 en hexadécimal est 45

Donc, le bloc de 32 bits dans l'adresse IPV6 qui représente la conversion en hexadécimal de l'adresse IPV4 est 9B69:3245

Travail demandé

1. Écrire une fonction `valide(ip)` qui permet de vérifier la validité d'une adresse IPV4 (True or False)
2. Écrire une fonction `classe(ip)` qui retourne la classe d'une adresse IP
3. Écrire une fonction `adresseip6(ip)` qui permet de convertir une adresse IP en V4 vers une adresse IPV6
4. Écrire la fonction `Genere()` qui permet de générer le fichier 'IPV6.txt'

Rappels :

- La fonction `bin(nb)` permet de convertir en binaire un nombre `nb` (`bin(155)` à `0b10011011`)
- La fonction `hex(nb)` permet de convertir un nombre décimal en hexadécimal (`hex(155)` à `0x9b`)

Exercice 3 : Opérations sur les listes

- Écrivez un script qui génère la liste des carrés et des cubes des nombres de 20 à 40.

1. Ecrire un programme en Python permettant de réaliser la différence de deux listes.

Exemple si :

```
L1 = [11, 3, 22, 7, 13, 23, 9]
L2 = [5, 9, 19, 23, 10, 23, 13]
```

Le programme renvoie la liste : `[11, 3, 22, 7]`

2. Ecrire un programme en Python permettant de réaliser la différence symétrique de deux listes `L1` et `L2`, c'est-à-dire la liste formée des éléments de `L1` qui ne sont pas dans `L2` et les éléments de `L2` qui ne sont pas dans `L1`.

Exemple si :

```
L1 = [11, 3, 22, 7, 13, 23, 9]
L2 = [5, 9, 19, 23, 22, 23, 13]
```

Le programme renvoie la liste : `[11, 3, 7, 5, 19]`

3. Etant donnée la liste des notes des élèves :

```
notes = [12, 04, 14, 11, 18, 13, 07, 10, 05, 09, 15, 08, 14, 16]
```

Ecrire un programme Python qui permet d'extraire de cette liste et créer une autre liste qui contient uniquement les notes au-dessus de la moyenne (les notes ≥ 10)

4. Écrire un algorithme en langage Python qui permet de renvoyer une liste formée des mots communs à deux textes. Exemple si `T1 = "Python est un langage de programmation"` et `T2 = "Python est orienté objet"`, le programme doit renvoyer la liste des mots communs `L = ['Python', 'est']`.

5. Créer une fonction Python nommée `toutEnMajuscule()` qui permet de transformer une liste de chaînes en une autre liste constituée de chaînes en majuscule.

Exemple si `L = ["Python", "est", "un", "langage", "de", "programmation"]`

La fonction doit retourner la liste `L2 = ["PYTHON", "EST", "UN", "LANGAGE", "DE", "PROGRAMMATION"]`

6. Écrivez un script qui crée automatiquement la liste des sinus des angles de 0° à 90°, par pas de 5°.
- Attention :** la fonction `sin()` du module `math` considère que les angles sont fournis en radians ($\text{radians} = \text{degrees} \times (\pi / 180)$).

Exercice 4 : Filtre de liste

Etant donné une liste de nombres entiers `L`, écrire un algorithme en Python permettant de renvoyer le couple de listes (`l_pair`, `l_impair`) où `l_pair` désigne la liste des entiers pairs de `L` et `l_impair` désigne la liste des entiers impairs de `L`.

Exemple pour :

```
L = [11, 3, 22, 14, 31, 18, 12, 7]
```

Le programme renvoie le couple de listes :

```
([22, 14, 18, 12], [11, 3, 31, 7])
```

Exercice 5 : Mélange de cartes

On considère les deux listes suivantes :

```
couleurs = ["Pique", "Trefle", "Carreau", "Coeur"]
```

```
valeurs = [2, 3, 4, 5, 6, 7, 8, 9, 10, "valet", "dame", "roi", "as"]
```

1. À partir de ces deux listes, générer une liste cartes contenant toutes les 52 cartes sous forme de chaînes de caractères. Utilisez un double parcours de ces listes, c'est-à-dire une double boucle `for`, ainsi que la méthode `str.append()`.
2. Importez la fonction `shuffle` de la librairie `random` grâce à l'instruction `from random import shuffle`. Cette fonction `shuffle()` permet de mélanger une liste. Mélangez alors la liste de vos cartes.
3. Créez ensuite quatre listes `joueur1`, `joueur2`, `joueur3` et `joueur4` qui correspondent à la distribution votre jeu de cartes mélangé à quatre joueurs différents.

Exercice 6 : Le crible d'Ératosthène

Le crible d'Ératosthène est une méthode pour calculer tous les nombres premiers inférieurs à une borne `n` donnée. On considère le code Python ci-dessous :

```
1. n = 1000 # On veut trouver tous les nombres premiers inferieurs a 1000
2. tab = n * [True]
3. tab[0] = False # 0 n'est pas premier
4. tab[1] = False # 1 n'est pas premier
5.
6. # Completer ci-dessous.
```

Compléter le code à l'endroit indiquer pour effectuer le crible sur le tableau `tab`. L'algorithme à appliquer est le suivant :

- pour tout entier i compris entre 2 (inclus) et n (exclus)
- si `tab[i]` est vrai alors :
 - pour tous les j , multiples de i entre $2 \times i$ (inclus) et n (exclus) :
 - mettre `tab[j]` à `False`

Après ce traitement, les indices de toutes les cases valant vrai sont des nombres premiers. Par exemple :

- On commence à parcourir avec i valant 2. `tab[2]` est `True`. On parcourt donc tout le tableau pour des valeurs de j valant 4, 6, 8, ...et on marque toutes ces cases à `False` (ce sont des multiples de 2, donc pas des nombres premiers).
- On passe à i valant 3. `tab[3]` est `True`. On parcourt donc tout le tableau pour des valeurs de j valant 6, 9, 12, ...et on marque toutes ces cases à `False`.
- On passe à i valant 4. `tab[3]` est `False` (marqué lors du traitement des multiples de 2), on passe à la suite.
- On passe à i valant 5. `tab[5]` est `True`. On parcourt donc tout le tableau pour des valeurs de j valant 10, 15, 20, ...et on marque toutes ces cases à `False`.

Dans l'exemple ci-dessus, on remarque que certains nombres sont marqués plusieurs fois à faux (par exemple 6 et 12 sont marqués lors du parcours du 2 et du 3).

1. Pour i valant 2, 3, 5, 7 : quelle est la première valeur de i qui ne vaut pas déjà faux lorsqu'on parcourt les multiples de i .
2. En déduire une modification du parcours des multiples évitant de remarquer plusieurs fois une valeur à faux.
3. En vous servant de l'observation précédente, est-il nécessaire de parcourir tous les entiers entre 2 et n ? peut-on s'arrêter avant ?

Exercice 1 : Tuples d'étudiants

1. Créez un programme qui demande à l'utilisateur d'entrer le prénom, le nom et le numéro de matricule d'un étudiant, puis stocke ces informations dans un tuple à trois éléments. Affichez ce tuple à l'écran.
2. L'intérêt d'utiliser un tuple dans ce cas réside dans le fait de ne pas pouvoir modifier les informations d'un étudiant, ce qui est plus sécurisé.
3. Modifiez votre programme de manière que l'utilisateur puisse entrer autant de saisies qu'il le souhaite. Le processus de saisie s'arrête lorsque l'utilisateur entre le mot "FIN". La liste des étudiants saisis sera stockée dans une liste de tuples. Utilisez une boucle `while`.
4. Affichez de manière conviviale tous les étudiants de votre liste construite au point précédent. Utilisez un parcours de liste.

Exercice 2 : Dictionnaire d'étudiants

Cet exercice correspond à une généralisation de l'exercice 1 dans le cas des dictionnaires.

1. Créez un programme qui demande à l'utilisateur d'entrer le prénom, le nom et le numéro de matricule d'un étudiant, puis stocke ces informations dans un dictionnaire. Les clés du dictionnaire correspondront aux noms des étudiants et ses valeurs seront des tuples à trois éléments (prénom, nom, matricule).
2. Affichez ce dictionnaire à l'écran. L'intérêt d'utiliser un dictionnaire dans ce cas réside dans le fait de pouvoir accéder aux informations des étudiants grâce à leurs noms.
3. Modifiez votre programme de manière que l'utilisateur puisse entrer autant de saisies qu'il le souhaite. Le processus de saisie s'arrête lorsque l'utilisateur entre le mot "FIN". Le dictionnaire des étudiants saisis sera stocké dans une liste de tuples. Utilisez une boucle `while`.
4. Affichez de manière conviviale tous les étudiants de votre dictionnaire construit au point précédent. Utilisez un parcours par clés de votre dictionnaire.
5. En utilisant la syntaxe `in`, déterminer si un étudiant du nom de "Kate" appartient à votre dictionnaire, et, si tel est le cas, renvoyez les informations cet étudiant.
6. Déterminez si le numéro de matricule 12345678 existe dans votre dictionnaire, et si tel est le cas, renvoyez les informations de l'étudiant correspondant. Utilisez un parcours clé-valeur de votre dictionnaire.
7. Dans l'état actuel de votre programme, il est impossible d'enregistrer deux étudiants qui portent le même nom. Pourquoi cela ? Modifier votre programme du point 2 de manière à pouvoir enregistrer des étudiants portant le même nom. Pour cela, les clés de votre dictionnaire ne devront plus

correspondre uniquement au nom des étudiants : mais attention, les clés ne peuvent pas être des listes.

Exercice 3 : Un dictionnaire français-anglais

1. Créez un dictionnaire qui contient un certain nombre de mots en français et comme valeurs la traduction de ces mots en anglais. Ce dictionnaire fait donc office de traducteur français-anglais.
2. Ajouter à votre dictionnaire le couple clé-valeur "cerveau" : "brain".
3. Recherchez si votre dictionnaire contient la traduction du mot "cerveau", et si tel est le cas, affichez sa traduction anglaise.
4. Effectuez un parcours par clés de votre dictionnaire et affichez chaque mot avec sa définition.
5. Écrire le contenu du dictionnaire dans un fichier nommé "fr.dic". Le contenu du fichier suivra le modèle suivant : On écrira le mot en français suivi de deux points, ensuite la traduction anglaise de ce mot.
6. Ouvrez le fichier créé au point précédent (point 5) et à l'aide d'un parcours de boucle, affichez son contenu à l'écran.
7. Créez un nouveau dictionnaire dont les clés et valeurs correspondent aux valeurs et aux clés de votre dictionnaire précédent. Ainsi, votre nouveau dictionnaire fera office de traducteur anglais-français au lieu de français-anglais. Pour construire ce nouveau dictionnaire, utilisez un parcours clé-valeur de votre dictionnaire initial.
8. Écrire le contenu du dictionnaire dans un fichier dénommé "en.dic" selon le modèle décrit au point.
9. Recherchez si votre dictionnaire contient la traduction du mot "brain".
10. Recherchez si votre dictionnaire contient la valeur "cerveau" et si tel est le cas, affichez sa clé correspondante.
11. Effectuez un parcours clé-valeur de votre dictionnaire et affichez chaque mot avec sa définition.
12. Modifiez votre dictionnaire de départ de sorte que les valeurs ne soient plus des simples mots anglais, mais des listes de mots correspondant autant de traductions possibles de vos clés.
13. Ajouter à votre nouveau dictionnaire le couple clé-valeur "chemin" : ["path", "way"].
14. Reprendre les instructions du point 5 et 6 pour la nouvelle structure du dictionnaire. Dans le cas où un mot possède plusieurs traductions, on les séparera par une virgule.
15. Par exemple pour le cas de chemin, on écrira dans le fichier : chemin:path,way et pour brain on écrira brain:cerveau.
16. Recherchez la deuxième traduction du mot "chemin".

17. Effacez la clé "chemin" et sa valeur correspondante de votre dictionnaire (Utilisez la fonction del).

Exercice 4 : Mesure de température

Les chercheurs étudiant les changements climatiques réalisent des observations dans certains jours de l'année. Les degrés de température proviennent de capteurs. Une observation sera une liste de 3 valeurs [jour, mois, degré].

Par exemple, une température de 20° C enregistrée le 13 février correspond à la liste [13, 2, 20].

Pour simplifier, on supposera que toutes les observations sont enregistrées pendant des années non bissextiles (le nombre de jours du mois de février d'une année non bissextile est égale à 28).

On suppose que les listes l_jours et l_mois suivantes sont préalablement définies :

Écrire une fonction **dict_mois_jours** qui, à partir des deux listes **l_jours** et **l_mois**, retourne un dictionnaire associant chaque mois à son nombre de jours.

1. Écrire une fonction **dict_ord_mois_jours** retournant un dictionnaire qui associe à chaque mois de la liste mois, son numéro dans l'année.
2. Écrire une fonction **observation_valide** qui vérifie si une observation est valide.

Une observation est valide si elle contient exactement 3 éléments :

- Le jour (1 à 31 selon le mois)
- Le mois (1 à 12)
- Le degré (entre -50 et 50)

3. Écrire une fonction **decire_observation** qui retourne une description textuelle d'une observation. Par exemple [14, 2, 10] donnera "Observation : 14 février, Température : 10°C"
4. Écrire une fonction **importer_observations** qui permet d'importer les observations depuis le fichier "observation.txt" se trouvant à la racine de cet exercice. Chaque ligne du fichier est au format : "jour mois degré" et retourne une liste d'observations valides.
5. Afficher la description textuelle de toutes les observations importées.
6. Écrire une fonction **numero_jour** qui retourne le numéro du jour dans l'année pour une observation. Par exemple, l'observation [14, 2, 10] correspond au 45e jour de l'an (14 février).
7. Écrire une fonction **avant** qui prend en paramètre deux observations obs1 et obs2 et qui retourne True si obs1 est mesuré avant obs2, False sinon.
8. Écrire une fonction **après** qui prend en paramètre deux observations obs1 et obs2 et qui retourne True si obs1 est mesuré après obs2, False sinon.
9. Écrire une fonction **moyenne_temp** qui calcule la température moyenne des observations valides.
10. Écrire une fonction **compter_observations_par_mois** qui retourne un dictionnaire associant chaque mois avec son nombre d'observations valides.

11. Implémenter les fonctions `mois_plus_froid` et `mois_plus_chaud` permettant respectivement de trouver le mois avec la température la plus froide et celle avec la température la plus chaude.
12. Sauvegarde dans un fichier texte chaque mois avec la moyenne de ses températures. Chaque ligne du fichier sera au format : "mois température_moyenne".